

Yohan Leconte

TP PostgreSQL

1)b) PostgreSQL en fonction :

```
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2025-11-18 09:52:30 CET; 40s ago
 Invocation: 10ce1655deaf4fd78d9fc2bf40215be9
   Main PID: 30571 (code=exited, status=0/SUCCESS)
   Mem peak: 1.8M
   CPU: 4ms
```

```
Nov 18 09:52:30 debian systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Nov 18 09:52:30 debian systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

c) Le port par défaut utilisé est 5432.

```
~
yohan@debian:~/ldiff$ sudo apt install net-tools
net-tools is already the newest version (2.10-1.3).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 92
yohan@debian:~/ldiff$ sudo netstat -ntlp | grep 5432
tcp        0      0 127.0.0.1:5432        0.0.0.0:*           LISTEN      31605/postgres
tcp6       0      0 :::1:5432             :::*                LISTEN      31605/postgres
yohan@debian:~/ldiff$
```

d) Connexion au service PostgreSQL en utilisateur 'postgres' :

```
user@deb:~$ su
Password:
root@deb:/home/user# su postgres
postgres@deb:/home/user$ psql
psql (17.6 (Debian 17.6-0+deb13u1))
Type "help" for help.

postgres=# \q
postgres@deb:/home/user$
```

e) Base de données « testdb » (voire ligne 4) :

Name	Owner	Encoding	Locale Provider	List of databases			ICU Rules	Access privileges
				Collate	Ctype			
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres + postgres=CTc/postgres
template1	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres + postgres=CTc/postgres
testdb (4 rows)	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			

f) On quitte PostgreSQL avec « \q ».

2) a) Connexion à la base de données testdb :

```
postgres=# \q
postgres@deb:/home/user$ psql testdb
psql (17.6 (Debian 17.6-0+deb13u1))
Type "help" for help.

testdb=# █
```

b) Table employees :

```
testdb=# CREATE TABLE employees (id SERIAL PRIMARY KEY, name VARCHAR(100), position VARCHAR(50), salary
INTEGER);
CREATE TABLE
testdb=# \d employees
              Table "public.employees"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer        |           | not null | nextval('employees_id_seq'::regclass)
 name         | character varying(100) |           |          |
 position     | character varying(50) |           |          |
 salary       | integer        |           |          |
Indexes:
    "employees_pkey" PRIMARY KEY, btree (id)

testdb=# █
```

d) Etat de la table après insertion des entrées données (on sélectionne la table complète) :

```
testdb=# INSERT INTO employees (name, position, salary) VALUES ('John Doe', 'Manager', 60000), ('Jane S
mith', 'Developer', 55000), ('Emily Johnson', 'Designer', 50000);
INSERT 0 3
testdb=# SELECT * FROM employees;
 id |      name      | position | salary
-----+-----+-----+-----
  1 | John Doe       | Manager  |  60000
  2 | Jane Smith     | Developer |  55000
  3 | Emily Johnson  | Designer |  50000
(3 rows)

testdb=# █
```

f) Après suppression de l'entrée avec id = 2 :

```
testdb=# DELETE FROM employees WHERE id = 2;
DELETE 1
testdb=# SELECT * FROM employees;
 id |      name      | position | salary
-----+-----+-----+-----
  1 | John Doe       | Manager  |  60000
  3 | Emily Johnson  | Designer |  50000
(2 rows)

testdb=#
```

3) a) Création d'un utilisateur « dbuser » avec mot de passe :

```
testdb=# CREATE USER dbuser WITH PASSWORD 'password123';
CREATE ROLE
testdb=# \du
```

List of roles	
Role name	Attributes
dbuser	
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS

b) Ajout des privilèges de dbuser sur la base testdb (ligne 4, privilèges d'accès) :

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
template1	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			postgres=CTc/postgres +
testdb	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
								postgres=CTc/postgres +
								=Tc/postgres +
								postgres=CTc/postgres +
								dbuser=CTc/postgres

(4 rows)

c) Droits de suppression retirés, connexion sous dbuser et tentative d'utiliser DELETE (échec) :

```
testdb=# REVOKE DELETE ON employees FROM dbuser;
REVOKE
testdb=# \q
```

```
postgres@debian:/home/yohan/ldiff$ psql -h 127.0.0.1 -d testdb -U dbuser
Password for user dbuser:
psql (17.6 (Debian 17.6-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.
```

```
testdb=> DELETE FROM employees WHERE id=1;
ERROR: permission denied for table employees
testdb=> █
```

d) Modification du mot de passe utilisateur pour dbuser, et reconnexion (l'ancien mot de passe ne fonctionne effectivement plus) :

```
testdb=> ALTER USER dbuser WITH PASSWORD 'JeSuisTropFort';
ALTER ROLE
testdb=> \q
postgres@deb:/home/user$ psql -h 127.0.0.1 -d testdb -U dbuser
Password for user dbuser:
psql (17.6 (Debian 17.6-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

testdb=> █
```

```
postgres@deb:/home/user$ psql postgres
psql (17.6 (Debian 17.6-0+deb13u1))
Type "help" for help.

postgres=# ALTER USER dbuser RENAME TO newuser;
ALTER ROLE
postgres=#
```

```
postgres@deb:/home/user$ psql -h 127.0.0.1 -d testdb -U newuser
Password for user newuser:
psql (17.6 (Debian 17.6-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

testdb=> █
```

```
postgres=# \q
postgres@deb:/home/user$ psql testdb
psql (17.6 (Debian 17.6-0+deb13u1))
Type "help" for help.

testdb=# DROP TABLE IF EXISTS employees;
DROP TABLE
testdb=# \d
Did not find any relations.
testdb=#
```

```
postgres@deb:/home/user$ psql postgres
psql (17.6 (Debian 17.6-0+deb13u1))
Type "help" for help.

postgres=# DROP DATABASE IF EXISTS testdb;
DROP DATABASE
postgres=#
```

Name	Owner	Encoding	Locale	Provider	List of databases		Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	postgres	UTF8	libc		en_US.UTF-8	en_US.UTF-8					
template0	postgres	UTF8	libc		en_US.UTF-8	en_US.UTF-8					=c/postgres + postgres=Ctc/postgres
template1	postgres	UTF8	libc		en_US.UTF-8	en_US.UTF-8					=c/postgres + postgres=Ctc/postgres

(3 rows)

c) Création de la BDD proxyma :

yohan@debian: ~/ldiff

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
proxyma	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
template1	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			postgres=CTc/postgres +
								=c/postgres +
								postgres=CTc/postgres

(4 rows)

d) Création d'un utilisateur dédié pour LemonLDAP ::NG (lemonldap) :

```
postgres=# CREATE USER lemonldap WITH PASSWORD 'lemonldap';
CREATE ROLE
postgres=# \du
```

List of roles	
Role name	Attributes
lemonldap	
newuser	
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS

e) Droits octroyés à lemonldap, il est maintenant propriétaire de la base de données :

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
proxyma	lemonldap	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
template1	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			postgres=CTc/postgres +
								=c/postgres +
								postgres=CTc/postgres

(4 rows)

5) a) Privilèges sur la table de configuration de LemonLDAP ::NG et des droits :

```
proxyma=# \z lmconfig
```

Access privileges				
Schema	Name	Type	Access privileges	Column privileges
public	lmconfig	table	postgres=arwdDxtm/postgres+ lemonldap=arwd/postgres	

(1 row)

b) Liste des index de la table persistentsessions :

Schema	Name	Type	Owner	Table
public	lmconfig_pkey	index	postgres	lmconfig
public	persistentsessions_pkey	index	postgres	persistentsessions
public	sp1	index	postgres	persistentsessions
public	suidp1	index	postgres	persistentsessions

(4 rows)

c) Test de connexion à la BDD avec le compte de lemonldap (succès) :

```
postgres@debian:/home/yohan/ldiff$ psql -h 127.0.0.1 -d proxyma -U lemonldap
Password for user lemonldap:
psql (17.6 (Debian 17.6-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.
```

```
proxyma=> \d
               List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | lmconfig       | table | postgres
public | persistensessions | table | postgres
(2 rows)
```

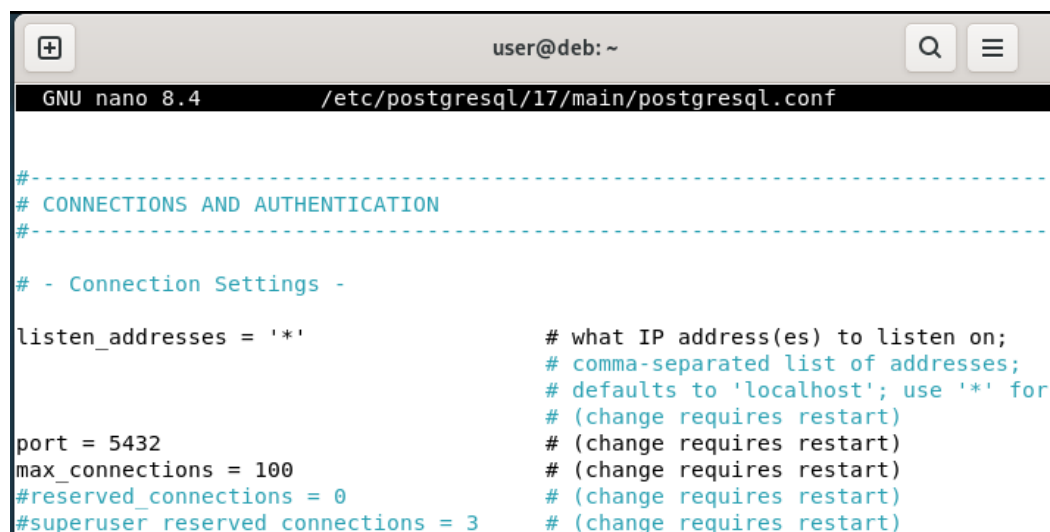
d) Postgres ne devient pas automatiquement accessible depuis l'extérieur même si la connexion locale fonctionne. On ne peut donc joindre le port 5432 de PostgreSQL de l'extérieur.

Avec recherchant le port 5432, on voit 127.0.0.1:5432, PostgreSQL écoute seulement en local (car 127.0.0.1 est l'interface locale).

e) Dans /etc/postgresql/17/main/pg_hba.conf, on ajoute la route par défaut pour autoriser l'utilisateur lemonldap à se connecter à la base proxyma depuis l'extérieur.

```
# Allow replication connections from localhost, by a user with the
# replication privilege.
local   replication    all                                     peer
host    replication    all             127.0.0.1/32          scram-sha-256
host    replication    all             ::1/128              scram-sha-256
host    proxyma        lemonldap       0.0.0.0/0             md5
```

f) Modification de postgresql.conf pour écouter sur l'IP de la VM (ligne listen_addresses = 'localhost') :



```
GNU nano 8.4 /etc/postgresql/17/main/postgresql.conf

#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = ''                                # what IP address(es) to listen on;
                                                    # comma-separated list of addresses;
                                                    # defaults to 'localhost'; use '*' for
                                                    # (change requires restart)
port = 5432                                           # (change requires restart)
max_connections = 100                                # (change requires restart)
#reserved_connections = 0                            # (change requires restart)
#superuser_reserved_connections = 3                  # (change requires restart)
```

On peut aussi indiquer l'IP de la VM précisément en l'assignant à la variable `listen_addresses`, mais ici on autorise toutes les interfaces.

Après avoir redémarré PostgreSQL :

```
user@deb:~$ sudo nano /etc/postgresql/17/main/pg_hba.conf
user@deb:~$ sudo nano /etc/postgresql/17/main/postgresql.conf
user@deb:~$ sudo systemctl restart postgresql
user@deb:~$ sudo netstat -tulpen | grep 5432
tcp        0      0 0.0.0.0:5432          0.0.0.0:*            LISTEN
112        18004      3103/postgres
tcp6       0      0 :::5432              :::*                  LISTEN
112        18005      3103/postgres
user@deb:~$
```

PostgreSQL ouvre le socket 0.0.0.0:5432 maintenant au lieu de 127.0.0.1:5432 (ou `<IP_VM>:5432` si on avait assigné son adresse IP à la place). Désormais, le `monldap` peut se connecter depuis l'extérieur.

TP Redis

1) b) Redis démarré (active running status) :

```
Processing triggers for libc-bin (2.41-12) ...
yohan@debian:~$ sudo systemctl status redis-server
● redis-server.service - Advanced key-value store
   Loaded: loaded (/usr/lib/systemd/system/redis-server.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-11-18 14:45:29 CET; 15s ago
 Invocation: f2f367a43536413fb6f0913cc1481862
    Docs: http://redis.io/documentation,
          man:redis-server(1)
   Main PID: 3425 (redis-server)
      Status: "Ready to accept connections"
     Tasks: 6 (limit: 2283)
    Memory: 7.8M (peak: 8.1M)
       CPU: 75ms
    CGroup: /system.slice/redis-server.service
            └─3425 /usr/bin/redis-server 127.0.0.1:6379"

Nov 18 14:45:29 debian systemd[1]: Starting redis-server.service - Advanced key-value store...
Nov 18 14:45:29 debian systemd[1]: Started redis-server.service - Advanced key-value store.
```

c) Interface en ligne de commande de Redis :

```
Nov 18 14:45:29 debian systemd[1]: Started redis-server.service - Advanced key-value store.
yohan@debian:~$ redis-cli ping
PONG
```

d) Port par défaut de Redis : 6379.

```
root@
yohan@debian:~$ sudo netstat -ntlp | grep 6379
tcp        0      0 127.0.0.1:6379        0.0.0.0:*           LISTEN      3425/redis-server 1
tcp6       0      0 :::6379               :::*                 LISTEN      3425/redis-server 1
```


e) Ajout du mot de passe 'redis' pour accéder à la base Redis :

```
GNU nano 8.4 /etc/redis/redis.conf *
# Redis configuration file example.
requirepass redis
# Note that in order to read the configuration file, Redis must be
# started with the file path as first argument:
#
# ./redis-server /path/to/redis.conf

# Note on units: when memory size is needed, it is possible to specify
# it in the usual form of 1k 5GB 4M and so forth:
#
# 1k => 1000 bytes
# 1kb => 1024 bytes
# 1m => 1000000 bytes
# 1mb => 1024*1024 bytes
# 1g => 1000000000 bytes
# 1gb => 1024*1024*1024 bytes
#
# units are case insensitive so 1GB 1Gb 1gB are all the same.

##### INCLUDES #####

# Include one or more other config files here. This is useful if you
# have a standard template that goes to all Redis servers but also need
# to customize a few per-server settings. Include files can include
# other files, so use this wisely.
#
# Note that option "include" won't be rewritten by command "CONFIG REWRITE"
# from admin or Redis Sentinel. Since Redis always uses the last processed
# line as value of a configuration directive, you'd better put includes
# at the beginning of this file to avoid overwriting config change at runtime.
#
# If instead you are interested in using includes to override configuration
# options, it is better to use include as the last line.
#
# Included paths may contain wildcards. All files matching the wildcards will
# be included in alphabetical order.
# Note that if an include path contains a wildcard but no files match it when
# the server is started, the include statement will be ignored and no error will
# be emitted. It is safe, therefore, to include wildcard files from empty
# directories.
#
# include /path/to/local.conf
# include /path/to/other.conf
# include /path/to/fragments/*.conf
#

##### MODULES #####
```

On redémarre ensuite le serveur Redis avec le mot de passe maintenant requis pour y accéder.

2) a) Connexion à la BDD Redis (authentification requise) :

```
yohan@debian:~$ redis-cli
127.0.0.1:6379> auth redis
OK
127.0.0.1:6379>
```

b) Quelques vérifications :

```
127.0.0.1:6379> CONFIG GET loglevel
```

```
1) "loglevel"
```

```
2) "notice"
```

```
127.0.0.1:6379> PING
```

```
PONG
```

```
127.0.0.1:6379> monitor
```

```
OK
```

```
█
```

La configuration complète :

```
127.0.0.1:6379> CONFIG GET *
```

```
1) "repl-backlog-ttl"
2) "3600"
3) "zset-max-ziplist-entries"
4) "128"
5) "tls-ciphersuites"
6) ""
7) "slave-serve-stale-data"
8) "yes"
9) "shutdown-on-sigterm"
10) "default"
11) "tls-ciphers"
12) ""
13) "socket-mark-id"
14) "0"
15) "tcp-backlog"
16) "511"
17) "replica-priority"
18) "100"
19) "databases"
20) "16"
21) "tls-client-key-file-pass"
22) ""
23) "cluster-allow-pubsubshard-when-down"
24) "yes"
25) "client-query-buffer-limit"
26) "1073741824"
27) "aof_rewrite_cpulist"
28) ""
29) "hll-sparse-max-bytes"
30) "3000"
31) "aclfile"
32) ""
33) "cluster-slave-no-failover"
34) "no"
35) "tls-protocols"
36) ""
37) "timeout"
38) "0"
39) "cluster-require-full-coverage"
40) "yes"
41) "loglevel"
42) "notice"
43) "cluster-preferred-endpoint-type"
44) "ip"
45) "unixsocketperm"
46) "0"
47) "proc-title-template"
48) "{title} {listen-addr} {server-mode}"
49) "replica-serve-stale-data"
50) "yes"
```

3) a) Stockage d'une clef name de valeur 'John Doe' :

```
127.0.0.1:6379> GET name
```

```
"John Doe"
```

```
127.0.0.1:6379> █
```

b) Ajout d'éléments dans une liste « tasks » et affichage de son contenu :

```
127.0.0.1:6379> LPUSH tasks "Apprendre Redis" "Faire TP" "Rediriger documentaire" "lerebours"
(integer) 4
127.0.0.1:6379> LRange tasks 0 -1
1) "lerebours"
2) "Rediriger documentaire"
3) "Faire TP"
4) "Apprendre Redis"
127.0.0.1:6379>
```

c) Suivi du compteur counter :

```
127.0.0.1:6379> INCR counter
(integer) 1
127.0.0.1:6379> GET counter
"1"
127.0.0.1:6379> INCR counter
(integer) 2
127.0.0.1:6379> GET counter
"2"
127.0.0.1:6379> DECR counter
(integer) 1
127.0.0.1:6379> GET counter
"1"
127.0.0.1:6379> █
```

d) Suppression de la clef name :

```
127.0.0.1:6379> DEL name
(integer) 1
127.0.0.1:6379> GET name
(nil)
127.0.0.1:6379> █
```

e) Activité suivie de la base de données Redis :

```
127.0.0.1:6379> monitor
OK
1763475278.770956 [0 127.0.0.1:36310] "auth" "(redacted)"
1763475311.065017 [0 127.0.0.1:36310] "INCR" "counter"
1763475316.772055 [0 127.0.0.1:36310] "GET" "name"
1763475330.914570 [0 127.0.0.1:36310] "SET" "name" "yoyo"
1763475364.722909 [0 127.0.0.1:36310] "LRange" "tasks" "0" "-1"
```

```
yohan@debian:~$ redis-cli
127.0.0.1:6379> auth redis
OK
127.0.0.1:6379> INCR counter
(integer) 2
127.0.0.1:6379> GET name
(nil)
127.0.0.1:6379> SET name "yoyo"
OK
127.0.0.1:6379> LRange tasks 0 -1
1) "lerebours"
2) "Rediriger documentaire"
3) "Faire TP"
4) "Apprendre Redis"
127.0.0.1:6379> 
```

Les données de la BDD sont correctement sauvegardées.

TP Nginx

1) c) Le port 511 est ouvert et est associé au service "Nginx".

```
yohan@debian:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: en>
   Active: active (running) since Mon 2025-12-08 15:00:47 CET; 38s ago
  Invocation: 84af21b263ae4cda87421e572703820e
     Docs: man:nginx(8)
    Process: 17623 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_proc>
    Process: 17626 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (>
   Main PID: 17665 (nginx)
      Tasks: 2 (limit: 2283)
     Memory: 2.8M (peak: 5.2M)
        CPU: 27ms
    CGroup: /system.slice/nginx.service
            └─17665 "nginx: master process /usr/sbin/nginx -g daemon on; maste>
              └─17666 "nginx: worker process"

Dec 08 15:00:47 debian systemd[1]: Starting nginx.service - A high performance >
Dec 08 15:00:47 debian systemd[1]: Started nginx.service - A high performance w>
lines 1-17/17 (END)
```

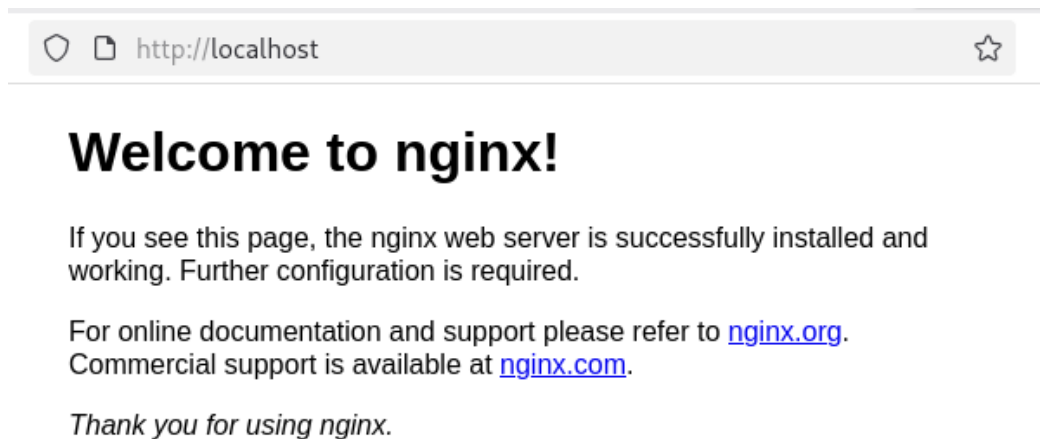
```
yohan@debian:~$ sudo netstat -ntlp | grep nginx
tcp        0      0 0.0.0.0:80          0.0.0.0:*           LISTEN
17665/nginx: master
tcp6       0      0 :::80              :::*                 LISTEN
17665/nginx: master
yohan@debian:~$ sudo ss -ptl4 | grep nginx
LISTEN 0      511      0.0.0.0:http        0.0.0.0:*    users:(("nginx",
pid=17666,fd=5),("nginx",pid=17665,fd=5))
yohan@debian:~$
```

Le port 80 (lié au service HTTP) de Nginx est ouvert.

d) HTTP GET demande la ressource entière (en-têtes + corps), tout le contenu est obtenu, alors qu'avec HTTP HEAD le serveur ne renvoie pas le corps mais uniquement les en-têtes (plutôt pour tester si une ressource existe, vérifier sa taille avant de la télécharger, vérifier si le contenu de la ressource a été modifié ou économiser simplement de la bande passante).

Avec HTTP/1.1, les connexions sont persistantes par défaut (voire "keep-alive") ce qui améliore fortement la performance car avec ces connexions, plusieurs requêtes peuvent passer sur une seule connexion TCP, tandis qu'en HTTP/1.0 il faut une nouvelle connexion TCP pour chaque ressource à regarder. La version 1.1 a d'autres optimisations importantes et des nouvelles fonctionnalités par rapport à la version 1.0.

2) a) Page par défaut de Nginx :



3) b) Rien ne change, les serveurs Nginx n'écoutent pas sur le port 80 mais 8080 et 8081. Le localhost continue donc d'afficher le site par défaut.

4) c) Sur <http://localhost> toujours la page par défaut.

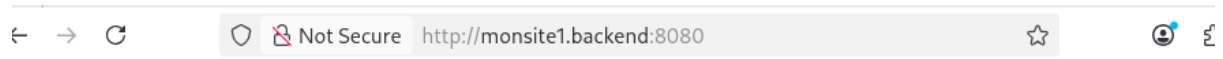
On a accès aux pages des Site1 et Site2 en utilisant les bons ports :
<http://localhost:8080> et <http://localhost:8081>.

d) Edition de /etc/hosts : on ajoute les lignes

127.0.0.1 monsite1.backend

127.0.0.1 monsite2.backend

e) Les sites Web depuis leur nouveau nom de domaine :



Bienvenue sur mon magnifique Site1 Nginx !



Bienvenue sur mon magnifique Site2 Nginx !

5) a) les sections principales pour 'nginx.conf' sont 'events' et 'http' :

```
events {
    worker_connections 768;
    # multi_accept on;
}

http {

    ##
    # Basic Settings
    ##

    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    server_tokens off; # Recommended practice is to turn this off

    # server_names_hash_bucket_size 64;
    # server_name_in_redirect off;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    ##
    # SSL Settings
    ##

    ssl_protocols TLSv1.2 TLSv1.3; # Dropping SSLv3 (POODLE), TLS 1.0, 1.1
    ssl_prefer_server_ciphers off; # Don't force server cipher order.

    ##
    # Logging Settings
    ##

    access_log /var/log/nginx/access.log;

    ##
    # Gzip Settings
    ##

    gzip on;

    # gzip_vary on;
    # gzip_proxied any;
    # gzip_comp_level 6;
    # gzip_buffers 16 8k;
    # gzip_http_version 1.1;
```

@@@

Les sections principales pour 'sites-available' sont 'server' et 'location' :

```
#
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # SSL configuration
    #
    # listen 443 ssl default_server;
    # listen [::]:443 ssl default_server;
    #
    # Note: You should disable gzip for SSL traffic.
    # See: https://bugs.debian.org/773332
    #
    # Read up on ssl_ciphers to ensure a secure configuration.
    # See: https://bugs.debian.org/765782
    #
    # Self signed certs generated by the ssl-cert package
    # Don't use them in a production server!
    #
    # include snippets/snakeoil.conf;

    root /var/www/html;

    # Add index.php to the list if you are using PHP
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        # First attempt to serve request as file, then
        # as directory, then fall back to displaying a 404.
        try_files $uri $uri/ =404;
    }

    # pass PHP scripts to FastCGI server
    #
    #location ~ \.php$ {
    #    include snippets/fastcgi-php.conf;
    #
    #    # With php-fpm (or other unix sockets):
```

18,0-1

13%

events{ } est le bloc contenant la gestion bas niveau des connexions réseaux.

http{ } a pour contenu tout ce qui concerne le protocole HTTP pour l'ensemble du serveur.

Dans server{ }, on retrouve l'unité de configuration du site web.

Enfin, location{ } contient les règles pour traiter une URL particulière et pour déterminer comment répondre.

b) Dans access.log, on peut savoir qui accède, à quel site, à quelle ressource, avec quel navigateur et voir si les sites ont répondu correctement ou si l'utilisateur a rencontré des erreurs : codes 200/301/404/500.

Error.log contient les erreurs de configuration, permissions, fichiers manquants, ports bloqués et indique lorsque le backend est indisponible.

6) a) Dans le fichier `/etc/nginx/nginx.conf`, on peut ajouter `"server_tokens off;"` dans la section `http{}`. Il est nécessaire de modifier ce fichier car notre système ne connaît pas les noms de domaines et le système ne sait pas où envoyer la requête.

b) On aurait pu éviter cette étape en écrivant <http://127.0.0.1:8080/> et <http://127.0.0.1:8081/> directement (comme testé en question 4-c).

c) Surveiller les fichiers journaux permet de détecter des attaques, de comprendre les erreurs et d'auditer ce que font les utilisateurs pour prévenir d'une activité suspecte.

d) Il est possible de sécuriser davantage l'installation de Nginx en limitant le nombre de connexions, en désactivant l'affichage de la version (question 6-a) et en restreignant l'accès à certains membres sur des périodes d'activité stratégiques par exemple.

e) Autres serveurs web que nous connaissons :

Apache HTTP Server (très modulable) et Caddy (où HTTPS est automatique).